# ELEC-H-310 Digital Electronics — Master Synthesis (Logic Circuits, Th05 → Th10)

This synthesis unifies six lectures covering the full chain from binary number representation to fully synchronous sequential logic circuits. The progression is logically cumulative: number encodings and Boolean algebra (Th05) provide the arithmetic and algebraic substrate; logic functions and K-Map optimization (Th06) supply the manipulation tools; combinatorial synthesis, hazards, and the bridge to sequential systems (Th07) introduce real-world circuit constraints and the notion of state; sequential circuit synthesis (Th08) formalises state tables, equivalence/fusion and the D Flip-Flop; asynchronous circuits (Th09) deal with race conditions and Moore vs. Mealy distinctions; finally synchronous logic circuits (Th10) introduce clocked memory elements (FFs) and excitation-function synthesis.

---

# Part I — Number representations and Boolean algebra (Th05)

## 1. Numbers representation

### Real (decimal) numbers — fixed point

- Example: **$(1372.6450)_{10}$** has integer part `1372` and decimal part `6450`, separated by a decimal point (`.` or `,`).
- Each digit has positional weight: $1 \cdot 10^3 + 3 \cdot 10^2 + 7 \cdot 10^1 + 2 \cdot 10^0 + 6 \cdot 10^{-1} + 4 \cdot 10^{-2} + 5 \cdot 10^{-3} + 0 \cdot 10^{-4}$.
- This is the **fixed point** representation; the number of digits (bits) for the integer/fractional parts is decided at circuit design time.
- Opposed to **floating point**, used for real numbers in most modern processors/ALUs (not covered here).

### General form for any base r

A number `N` in base `r` is written:

```
N_r = (a_n a_{n-1} ... a_0 . b_1 b_2 ... b_m)_r
```

with value:

```
N_r = Σ_{i=0..n} a_i · r^i  +  Σ_{j=1..m} b_j · r^{-j}  =  I + D
```

- `a_0` : **Least Significant Digit** (LSD); `a_n` : **Most Significant Digit** (MSD).
- For binary base: **LSB** and **MSB**. Knowing whether a word starts with MSB or LSB defines the read direction. By convention, MSB is on the left, LSB on the right (otherwise, ambiguity: $(1011)_2$ could mean 11 or 13).

## Useful bases

- **Decimal (r = 10)** — humans: $D \in \{0..9\}$.
- **Binary (r = 2)** — $D \in \{0,1\}$.
- **Octal (r = 8)** — $D \in \{0..7\}$.
- **Hexadecimal (r = 16)** — $D \in \{0..9, A..F\}$.
- Bases 2, 8, 16 are powers of two ($2^1$, $2^3$, $2^4$) and used in digital systems.

| Decimal | Binary (5b) | Octal | Hex |
|---|---|---|---|
| 0 | 00000 | 0 | 0 |
| 1 | 00001 | 1 | 1 |
| 2 | 00010 | 2 | 2 |
| 3 | 00011 | 3 | 3 |
| 4 | 00100 | 4 | 4 |
| 5 | 00101 | 5 | 5 |
| 6 | 00110 | 6 | 6 |
| 7 | 00111 | 7 | 7 |
| 8 | 01000 | 10 | 8 |
| 9 | 01001 | 11 | 9 |
| 10 | 01010 | 12 | A |
| 11 | 01011 | 13 | B |
| 12 | 01100 | 14 | C |
| 13 | 01101 | 15 | D |
| 14 | 01110 | 16 | E |
| 15 | 01111 | 17 | F |
| 16 | 10000 | 20 | 10 |

## 2. Base conversions

### Method of successive divisions — Dec→Bin (integer)

For 4-bit `N`, applying recursive factoring shows the LSB is `(N - a₀)/2`'s remainder, etc. Each division gives one bit, starting from the LSB.

Example: $\mathbf{(245)_{10}} \rightarrow \mathbf{(?)_2}$

| Dividend | ÷2 | Remainder |
|---|---|---|
| 245 | ÷2 | 1 |
| 122 | ÷2 | 0 |
| 61 | ÷2 | 1 |
| 30 | ÷2 | 0 |
| 15 | ÷2 | 1 |
| 7 | ÷2 | 1 |
| 3 | ÷2 | 1 |
| 1 | ÷2 | 1 |
| 0 | | |

Read remainders bottom-to-top: $\mathbf{(245)_{10} = (11110101)_2}$.

### Successive multiplications — Dec→Bin (decimal part)

Example: $\mathbf{(0.345)_{10}} \rightarrow \mathbf{(?)_2}$

| Value | ×2 | Bit |
|---|---|---|
| .345 | ×2 = .690 | 0 |
| .690 | ×2 = 1.380 | 1 |
| .380 | ×2 = .760 | 0 |
| .760 | ×2 = 1.520 | 1 |
| .520 | ×2 = 1.040 | 1 |
| .040 | ×2 = .080 | 0 |
| .080 | ×2 = .160 | 0 |
| .160 | ×2 = .320 | 0 |
| .320 | ×2 = .640 | 0 |
| .640 | ×2 = 1.280 | 1 |

Read top-to-bottom (10 digits): $\mathbf{(0.0101100001)_2}$. Stopping criterion is the fixed-point precision target.

## Bin → Dec (both parts)

Multiply each bit by its weight and sum:

- $(11110101)_2$ = 128+64+32+16+0+4+0+1 = **245**.
- $(.0101100001\ldots)_2$ = 1/4 + 1/16 + 1/32 = 11/32 = **0.34375**.

## Dec ↔ Octal and Dec ↔ Hex

- $(245)_{10} \rightarrow (?)_8$: successive ÷8 gives remainders 5, 6, 3 → $\mathbf{(365)_8}$. Reverse: 5·1 + 6·8 + 3·64 = 245.
- $(245)_{10} \rightarrow (?)_{16}$: ÷16 gives 5, F → $\mathbf{(F5)_{16}}$.
- $(A1F.1C)_{16}$ → 10·256 + 1·16 + 15 + 1/16 + 12/256 = **2591.1093…**

## Power-of-2 conversions via grouping

For bases 2, 8, 16, group binary digits into packs of **3 bits (octal)** or **4 bits (hex)**, starting from the LSB. Pad with leading zeros if needed.

Example: $\mathbf{(011110101)_2}$ → group 3 bits: 011 110 101 = $\mathbf{365_8}$; group 4 bits (pad): 0000 1111 0101 = $\mathbf{0F5_{16}}$.

Reverse from hex: **$(1F2)_{16}$** = 0001 1111 0010 → group 3 bits: 111 110 010 = **$762_8$**; decimal **498**.

### Arbitrary base conversions

Best practice: route through base 10. To convert `N_p → X_q`, do `N_p → N_{10} → X_q`.

Example: **$(25.34)_8$** → **$(?)_5$**: first to dec = $(21.4375)_{10}$; then to base 5 (successive ÷5 / ×5) → **$(41.204320)_5$**.

## 3. Arithmetic operations

### Binary addition of two 1-bit words (half-adder)

| a | b | $s_1$ (carry) | $s_0$ (sum) |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 |

This specification implements the **half-adder**.

### Three-bit addition (full-adder)

| a | b | c | $s_1$ | $s_0$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 1 | 1 |

This is the **full-adder**.

### N-bit ripple-carry adder

Sum of two n-bit words `a, b` produces an (n+1)-bit result:

```
c₀, s₀ = a₀ + b₀                    (half-adder)
cᵢ, sᵢ = aᵢ + bᵢ + cᵢ₋₁             (full-adder, ∀ i ∈ {1..n-1})
```

The circuit chains 1 half-adder + (n–1) full-adders in series → **ripple-carry adder**. Carry must propagate from LSB to MSB; this circuit becomes too slow for large n (long critical path).

### Addition example, 8 bits

`s = 236 + 170 = 406`

| Bit | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|
| Carry | | 1 | 1 | 1 | 0 | 1 | 0 | 0 | 0 |
| a | | 1 | 1 | 1 | 0 | 1 | 1 | 0 | 0 |
| b | | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 |
| Sum | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 0 |

Result needs 9 bits → **overflow** in fixed 8-bit registers; if stored in 8 bits, the result is truncated.

## 4. Negative numbers

### a) Sign and Magnitude (SM)

- 1 bit reserved for sign ( `0` = positive, `1` = negative); `n–1` bits for magnitude.
- 8-bit range: –127 .. +127 → only 255 distinct values, because **two zeros exist** ( `0 0000000` = +0 and `1 0000000` = –0).
- **Pros**: easy for humans, easy conversion.
- **Cons**: two representations of zero; subtraction needs sign comparison and conditional logic → slower, more silicon, more power.

### b) One's complement (C1)

- Motivation: turn subtraction into addition. `A − B = A + (−B) = A + (rᵐ − B)` ; in binary, just bit-wise inversion.
- Still suffers from **two zeros** ( `00000000` and `11111111` ).
- Computation example (decimal, m=3): `+87 − 53` → `87 + (1000−53)` = `87 + 947` = `1034` ; drop carry.

### c) Two's complement (C2)

- Reorganise: `B = (rᵐ − B) = ((rᵐ − 1) − B) + 1`. The first part is the **digit-wise complement**; in binary, it is implemented by an inverter.
- Adding 1 yields C2; **only one zero**, asymmetric range (e.g. 8-bit: –128 .. +127).
- Used by all modern computing machines.

| Decimal | 8-bit binary | C1 | C2 |
|---|---|---|---|
| 0 | 00000000 | (+)0 | 0 |
| 1 | 00000001 | 1 | 1 |
| 127 | 01111111 | +127 | +127 |
| 128 | 10000000 | –127 | –128 |
| 129 | 10000001 | –126 | –127 |
| 255 | 11111111 | (–)0 | –1 |

## Practical conversions to SM, C1 and C2

- Convert absolute value to binary (successive divisions).
- Pad with leading zeros to reach m bits.
- **SM**: keep m–1 bits; bit m is sign (0 positive, 1 negative).
- For negative numbers: invert all m bits → **C1**; add 1 → **C2**.
- For positive numbers: just use binary directly.

Example: encode $(57)_{10}$ and $(-23)_{10}$ on 8 bits:

| Value | Abs binary | SM | C1 | C2 |
|---|---|---|---|---|
| +57 | 0111001 | 0 0111001 | 0 0111001 | 0 0111001 |
| –23 | 0010111 | 1 0010111 | 1 1101000 | 1 1101001 |

### C2 arithmetic — overflow rule

To compute `C = A − B` in C2:

1. Convert positive `A` to C2 (just binary).
2. Convert negative `B` to C2 (invert + add 1 to its absolute value).
3. Perform addition.
4. **Check overflow**: ignore the carry-out only if the result `C` lies in $-r^{m-1} < C < r^{m-1} - 1$ (e.g. for 8 bits: –128 < C < 127) AND the **last two carry bits are equal**.

Example: `57 − 23` in 8-bit C2:

| | **bits 7..4** | **bits 3..0** | **dec** |
|---|---|---|---|
| Carry | 1 1111 | 001- | |
| A | 0011 | 1001 | +57 |
| B | 1110 | 1001 | –23 |
| C | 1 0010 | 0010 | +34 |

Overflow can be ignored (last two carry bits equal, result in range).

## 5. Boolean algebra

### Definition

A **Boolean algebra** is a quadruplet `{B, ´, ·, +}` where:

- `B` is a (here) two-element set.
- `´`, `·`, `+` are respectively **complement** (NOT), **AND**, **OR** operators.

We use **two-valued Boolean algebra** (Shannon 1938, after Huntington 1904):

- `B = {0, 1}` with `0 = FALSE`, `1 = TRUE`.
- `+` is **inclusive OR**, `·` is **AND**.
- These operators are **logical, not arithmetic**: `1 + 1 ≠ 2`.

### Truth tables for AND, OR, NOT

| x | y | x · y | x + y |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 |

| x | x′ |
|---|---|
| 0 | 1 |
| 1 | 0 |

### Six axioms

1. **Closure**: B is closed for `+` and `·`.
2. **Neutral element**: `0` is neutral for `+`, `1` is neutral for `·`.
3. **Commutativity** for `+` and `·`.
4. **Distributivity**: `·` distributes over `+` and vice-versa.
5. **Complement** of x exists.
6. There are 2 elements x, y in B such that `x ≠ y`.

**Associativity** follows: `(a+b)+c = a+(b+c)` and `(a·b)·c = a·(b·c)`.

### Basic theorems

| Th | Statement |
|---|---|
| Th1 — Idempotency | `x + x = x` and `x · x = x` |
| Th2 — Neutral op. | `x + 1 = 1` and `x · 0 = 0` |
| Th3 — Absorption | `x · (x + y) = x` |
| Th4 — Involution | `(x′)′ = x` |
| Th5 — Associativity | `(x+y)+z = x+(y+z)` and `(xy)z = x(yz)` |
| Th6 — De Morgan | `(x+y)′ = x′·y′` and `(x·y)′ = x′+y′` |
| Th7 — Consensus | `x·y + x′·z + y·z = x·y + x′·z` |

## Three proof methods

### a) Axioms / theorems

Apply step by step. Example proof of Th1:

```
x + x = (x + x) · 1                Ax2 (neutral)
      = (x + x)(x + x′)            Ax5 (complement)
      = x + x · x′                 Ax4 (distributivity)
      = x + 0                      Ax5 (complement)
      = x                          Ax2 (neutral)
```

### b) Truth tables

In two-valued Boolean algebra, theorems are combinatorial — compute both sides for all input combinations.

Distributivity demonstrated:

| x | y | z | y+z | x(y+z) | xy | xz | xy + xz |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |

De Morgan demonstrated:

| x | y | x+y | (x+y)′ | x′·y′ |
|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 1 | 0 | 0 |
| 1 | 1 | 1 | 0 | 0 |

**c) Duality principle**

Every result has its **dual**, obtained by interchanging `+ ↔ ·` and `0 ↔ 1`. If `S` is true, `S∗` is also true.

- Example: from `x + x = x` derive `x · x = x`.
- De Morgan: from `(x+y)′ = x′·y′` derive `(x·y)′ = x′+y′`.
- Generalised on n variables: `S(x₁,…,x⍰) → S∗(x₁,…,x⍰)`. Extension of De Morgan: `(x₁ + … + x⍰)′ = x₁′ · … · x⍰′` and dually.

---

# Part II — Logic functions, K-Maps and optimization (Th06)

## 1. Representing logic functions — three forms

### A. Truth tables (TTs)

- Provide function values for all input combinations.
- Each line = one input combination; each function output has its column.
- TTs uniquely represent a logic function. Multiple outputs → multiple TTs and multiple independent (concurrent) circuits.
- Sizes: `n` arguments → `2ⁿ` lines; `2^(2ⁿ)` distinct logic functions are possible.
- 1 var: 4 functions ( `F=0` , `F=1` , identity, inversion).
- 2 vars: 16 functions; 3 vars: 256 functions.

### All 16 logic functions of 2 variables

| # | 00 | 01 | 10 | 11 | Expression | Name |
|---|---|---|---|---|---|---|
| F0 | 0 | 0 | 0 | 0 | 0 | Zero |
| F1 | 0 | 0 | 0 | 1 | x·y | AND |
| F2 | 0 | 0 | 1 | 0 | x·y′ | Inhibition |
| F3 | 0 | 0 | 1 | 1 | x | Transfer |
| F4 | 0 | 1 | 0 | 0 | x′·y | Inhibition |
| F5 | 0 | 1 | 0 | 1 | y | Transfer |
| F6 | 0 | 1 | 1 | 0 | x·y′ + x′·y | XOR |
| F7 | 0 | 1 | 1 | 1 | x + y | OR |
| F8 | 1 | 0 | 0 | 0 | (x + y)′ | NOR |
| F9 | 1 | 0 | 0 | 1 | x·y + x′·y′ | XNOR (=) |
| F10 | 1 | 0 | 1 | 0 | y′ | Complement |
| F11 | 1 | 0 | 1 | 1 | x + y′ | Implication |
| F12 | 1 | 1 | 0 | 0 | x′ | Complement |
| F13 | 1 | 1 | 0 | 1 | x′ + y | Implication |
| F14 | 1 | 1 | 1 | 0 | (x·y)′ | NAND |
| F15 | 1 | 1 | 1 | 1 | 1 | One |

## B. Algebraic expressions

#### Standard canonical form — Sum of Minterms (SoM / CDNF)

- A **minterm** is a product term in which each of the n variables appears exactly once, for which the function equals `1`.
- The **Canonical Disjunctive Normal Form (CDNF)** is the OR of all minterms.

Example with 3 variables (function = 1 for `x = 0`, regardless of y, z):

| x | y | z | F |
|---|---|---|---|
| 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 0 |

`F = x′y′z′ + x′y′z + x′yz′ + x′yz`

**Standard canonical form — Product of Maxterms (PoM / CCNF)**

- Built from the lines where `F = 0`. Take the AND of summed inverted terms (one per `0` line).
- Example: `F = (x′ + y + z)(x′ + y + z′)(x′ + y′ + z)(x′ + y′ + z′)`.
- Derivation: `F′ = xy′z′ + xy′z + xyz′ + xyz`, then `F = (F′)′` and apply De Morgan.

**Decimal-equivalent notation**

Pick MSB/LSB convention (here MSB = x, LSB = z).

- Sum of minterms: `Σm(0,1,2,3)`.
- Product of maxterms: `ΠM(4,5,6,7)`.
- The set M is the complement of m (over `2ⁿ` decimal equivalents).

**Summary**

- **CDNF (SoM)** = OR of products (ANDs) where F = 1.
- **CCNF (PoM)** = AND of summed inverted terms where F = 0.
- Each canonical form is unique for a given TT; switching is mechanical and safe; physical implementation cost may differ.

### C. Schematics — basic gate symbols

- **AND, NAND, OR, NOR, XOR, XNOR, NOT (INV)** — circles on inputs/outputs denote inversion.
- **Functionally Complete Set (FCS)** — a set of operators that can express all Boolean functions by combination only.
- **Singleton FCS**: `FCS₂(NAND)` and `FCS₃(NOR)` — one gate suffices to build the rest. Useful in practical IC manufacturing.
- A SoP circuit features an **AND plane** (parallel AND gates) feeding a single OR gate → **two-level logic**. Practical circuits use **multi-level logic**.

## 2. Transformation of logical expressions

### Non-standard canonical form

By inspection, the previous TT yields `F = x′` directly (since F = 1 ⇔ x = 0). Equivalent to SoM after axiom transformations:

```
F = x′y′z′ + x′y′z + x′yz′ + x′yz
  = x′y′(z′ + z) + x′y(z′ + z)
  = x′y′ + x′y
  = x′(y′ + y) = x′
```

### Logic optimization — PPACT motivation

Both expressions implement the same function, but the simpler one is better in:

- **Performance** (smaller delay).
- **Power** (fewer transistors).
- **Area** (less silicon).
- **Cost** (smaller die).
- **Temperature** (lower dissipation).

Together: **PPACT** constraints. Logic optimization is central to logic synthesis (translation of specs into equations); typically automated.

### From SoM back to a Sum of Products (SoP)

After optimization, terms are no longer minterms (some variables drop out):

```
F(x,y,z) = x′ + y′z + xz′
```

- 1st term lacks y, z; 2nd lacks x; 3rd lacks y.

### Term expansion

To recover the SoM from a generic SoP:

```
xy′ = xy′ · 1 = xy′(z + z′) = xy′z + xy′z′
```

For a maxterm (using `x + 0 = x` and `xx′ = 0`):

```
x + y′ = x + y′ + zz′ = (x + y′ + z)(x + y′ + z′)
```

### SoP and netlist representation

A logic function written as SoP can be expressed using gate operators:

```
F = OR(AND(x,z), AND(x,NOT(y),NOT(z)), AND(NOT(x),y,NOT(z)))
```

This formalism is used by EDA tools to produce **netlists** — text files describing logic functionality of digital systems with millions of gates.

## 3. Karnaugh maps (K-Maps)

### n-cubes

- An **n-cube** is an n-dimensional hypercube with $2^n$ vertexes; each vertex is associated with an n-bit binary word.
- **Hamming Distance (HD)** between two equal-length strings = the number of differing positions. Adjacent vertexes of an n-cube edge have **HD = 1**.

### K-Map definition

- A **K-Map** is a TT reorganised like an n-cube and drawn in 2D.
- It is just a graphical TT — no new info, only re-arranged.
- Built from a TT (copy values), from an algebraic expression (after term expansion), or directly from minterms.

### K-Map sizes and conventions

- **2 vars** F(a,b): 2×2 grid with rows a and columns b. Convention: **a is MSB, b is LSB**.
- **3 vars** F(a,b,c): 2×4 grid, columns are bc in order **00, 01, 11, 10** (Gray code) to maintain adjacency.
- **4 vars** F(a,b,c,d): 4×4 grid, both rows ab and columns cd use Gray-code ordering.
- **5 vars**: two 4-variable K-Maps placed side-by-side (one for `a=0`, one for `a=1`).
- **6 vars**: four 4-variable K-Maps in a grid (for $X_5X_4 \in \{00, 01, 10, 11\}$).

### Sub-cubes

- A **sub-cube** of `2^m` vertexes belongs to an edge or face of an n-cube; for the remaining `n - m` variables, the function has the same value.
- Formally: `F = F_f · F_v` where `F_f` is the **fixed part** and `F_v` represents all combinations of the free variables (so `F_v` simplifies to 1).
- Logical interpretation: F = 1 for the fixed part AND for all combinations of variables in the variable part.

### Examples

- **2-input K-Map, sub-cube of size 2**: covers `a´` (entire row a=0) or `a` (row a=1); similarly for b.
- **3-input K-Map, sub-cubes of size 2**: `b´c´, b´c, bc, bc´` along columns; `a´c, ab´, ab, a´c´` other groupings.
- **3-input K-Map, sub-cubes of size 4**: `a, a´, b, b´, c, c´`.
- **4-input K-Map, sub-cubes of size 4**: `a´c´ ∀b,d`; `cd´ ∀a,b`; `ab´ ∀c,d`.
- **4-input K-Map, sub-cubes of size 8**: cover an entire half (variable = 0 or 1) such as `c´ ∀a,b,d`.

## 4. Logic function optimization using K-Maps

### Why optimize

Implementing in standard canonical form (SoM) yields many AND gates with fixed input count and one large OR. Transforming SoM into a smaller equivalent expression reduces gate count, area, power, delay → **PPACT minimisation**.

## Optimization methods

- **By axioms / theorems** — quality depends on the designer; complexity scales exponentially.
- **Karnaugh maps (Karnaugh, 1953)** — graphical method, very efficient up to ~5 variables.
- **Quine–McCluskey (Quine 1952, McCluskey 1956)** — systematic, scalable, automatable; basis of modern CAD.

## Key sub-cube definitions

- **Prime Implicant (PI)** — biggest sub-cube not contained in any other sub-cube of the K-Map.
- **Essential Prime Implicant (EPI)** — single PI that covers at least one `1` not covered by any other PI.
- Optimised function = OR of all EPIs + the fewest PIs needed to cover the remaining 1s.
- For any non-standard expression, all terms are PIs: `F = PI₁ + PI₂ + … + PI?`.

## Simplification algorithm

1. Find all PIs (look at all maximal sub-cubes).
2. Isolate all EPIs (must be taken).
3. Cover remaining 1s with the fewest additional PIs.

Goal: smallest number of biggest sub-cubes → fewer AND gates of smaller width.

## Examples

### 3 variables: `F = Σm(1,3,7)`

Two PIs of size 2: `PI₁ = a´c`, `PI₂ = bc`. Both essential (red covers 001 only, green covers 111 only). **F = a′c + bc**.

### Superposition is mandatory

Sub-cubes can (and should) overlap to keep them as large as possible. For the same function, taking smaller non-overlapping sub-cubes yields suboptimal expressions like `F = ac´ + abc` or `F = bc + a´b´c`, while superposition gives `F = a´c + bc`.

### Removing redundant PIs

If all 1s are already covered by certain PIs (e.g., `a´c` and `ab`), do not include the redundant `bc`. Default: provide the optimised function unless instructed otherwise.

#### 4 variables

`F = c′d′ + bd′ + abc` — all PIs essential.

#### 5 variables

Sub-cubes can span both K-Maps (3D adjacency). Example: `a′bc′de′ + abc′de′ = bc′de′`.

End result for the example: `F = a′c′ + ce + b′de + bc′de′` (last PI `de′` was non-essential).

## 5. Incompletely specified logic functions

### “Don’t cares” vs “don’t happens”

- **Don’t care (–)**: input combination may occur, but we do not care about the output.
- **Don’t happen**: input combination cannot occur (e.g., top + bottom elevator sensor both true).
- In practice, both are written `–`.

### New set B and usage

- Now `B ∈ {0, 1, –}`.
- During K-Map simplification, set each `–` to `0` or `1` whichever yields a larger sub-cube.
- Caution: blindly setting all `–` to `1` may add extra terms; pick wisely.

### “Don’t cares” on inputs

A sub-cube whose fixed part is, e.g., $x_2 = 1$ AND $x_3 = 0$, with $x_1$ variable, can be written $-x_2x_3'$; the dash reminds you of the variable. Equivalent to expanding $x_1x_2x_3' + x_1'x_2x_3'$.

### Examples

3-input function `F(a,b,c) = Σm(1,5,7) + Σd(3)` — depending on choice for the “–”: with `–` = 0, `F = b′c + ac`; with `–` = 1, `F = c`. The latter is clearly better.

4-input function `F(a,b,c,d) = Σm(0,6,8,12,14,15) + Σd(2,4,7)` — list of PIs: $x_2'x_1'$, $x_3x_2$, $x_4'x_1'$; only the first two are essential and cover the function: $F = x_2'x_1' + x_3x_2$.

---

# Part III — Combinatorial logic synthesis, hazards, intro to sequential (Th07)

## 1. Combinatorial logic synthesis from verbal specifications

### The synthesis problem

- Transform an informal (typically verbal) specification into a formal one (Boolean expressions, TT, K-Map, schematic).
- Verbal specs describe **what** the system should do at high abstraction; we must decide **how** it is done — in software: algorithm; in hardware: architecture.

### Formal pipeline

1. Establish the **truth table** — formal specification of the problem; **must be correct**, otherwise everything else is wrong.
2. Derive logic expressions (pen & paper or computer).
3. Optimise.
4. Implement as a logic circuit. Number of inputs fixes function size; number of outputs fixes the number of independent functions (each implemented as its own circuit, all running concurrently).

### Combinatorial logic — formal definition

`O = f(I)` where `i ∈ {0,…,n−1}`. Properties:

- Same input combination always produces the same output.
- **No memory.**
- Data flows from inputs to outputs through wires/gates; outputs are valid as long as inputs are valid.
- All inputs must be present at `0` or `1` — **hard switches** required (push-buttons that don't latch are excluded).

## 2. Synthesis of adder circuits

### Half-adder — derived equations

| a | b | $s_1$ | $s_0$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 |

$s_0 = a_i' b_i + a_i b_i' = a_i \oplus b_i$ ; $s_1 = a_i \cdot b_i$.

### Full-adder — derived equations

| a | b | c ($r_i$) | $s_1$ | $s_0$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 1 | 1 |

$s_0 = a_i \oplus b_i \oplus r_i$ ; $s_1 = a_i b_i + r_i b_i + r_i a_i$.

### N-bit ripple-carry adder

Same structure as in Th05: 1 half-adder for the LSB and (n–1) full-adders chained. Carry must propagate from LSB to MSB → poor scalability.

### Going beyond ripple

The whole n-bit addition can also be specified as a single combinatorial problem with `2n` inputs and `n+1` outputs. For 2×2-bit words, the TT has 16 lines. The downside: combinatorial blow-up — number of TT lines grows as `2^(2n)`.

## 3. Circuits with multiple outputs

### Example 1 — 7-segment display

A 4-bit input drives 7 LEDs (one logic function `fᵢ` per segment; `fᵢ = 1` lights LED `i`).

| Decimal | $f_1$ | $f_2$ | $f_3$ | $f_4$ | $f_5$ | $f_6$ | $f_7$ |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 1 | 1 | 1 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| 2 | 1 | 1 | 0 | 1 | 1 | 1 | 0 |
| 3 | 1 | 1 | 1 | 1 | 0 | 1 | 0 |
| 4 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| 5 | 1 | 0 | 1 | 1 | 0 | 1 | 1 |
| 6 | 1 | 0 | 1 | 1 | 1 | 1 | 1 |
| 7 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |
| 8 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 9 | 1 | 1 | 1 | 1 | 0 | 1 | 1 |

For inputs 10–15 there are three options:

- **Solution 1: don't cares** — best simplification but possibly garbage on the display.
- **Solution 2: don't happens** — inputs 10–15 never occur (e.g., counter resets at 10); always correct.
- **Solution 3: explicit value** — print an "E" for any input > 9; uniform output for 10–15. May produce more complex (but predictable) logic.

### Example 2 — BCD → Excess-3 converter

| Decimal | BCD (8421) | Excess-3 (2421) |
|---|---|---|
| 0 | 0000 | 0000 |
| 1 | 0001 | 0001 |
| 2 | 0010 | 0010 |
| 3 | 0011 | 0011 |
| 4 | 0100 | 0100 |
| 5 | 0101 | 1011 |
| 6 | 0110 | 1100 |
| 7 | 0111 | 1101 |
| 8 | 1000 | 1110 |
| 9 | 1001 | 1111 |
| 10–15 | … | – – – – |

Excess-3 is a self-complementary code. Optimised functions (with don't cares for 10–15):

- $f_1 = x_1$
- $f_2 = x_4 + x_3x_2'x_1 + x_3'x_2$
- $f_3 = x_4 + x_3x_2 + x_3x_1'$
- $f_4 = x_4 + x_3x_1 + x_3x_2$

Decimal-equivalent compact form:

$$
\begin{aligned}
f_1 &= \Sigma m(1,3,5,7,9) + \Sigma d(10..15) \\
f_2 &= \Sigma m(2,3,5,8,9) + \Sigma d(10..15) \\
f_3 &= \Sigma m(4,6,7,8,9) + \Sigma d(10..15) \\
f_4 &= \Sigma m(5,6,7,8,9) + \Sigma d(10..15)
\end{aligned}
$$

# 4. Hazards

## The reality of delays

- Idealised circuits: zero gate/wire delay, instantaneous transitions.
- Real circuits: resistance, inductance, capacitance produce **gate switching time** `t = (t_LH + t_HL)/2`, plus wire delays (function of length).
- **Different paths may have different total delays.**

## Glitches / hazards

If two AND gates feeding a final OR switch in opposite directions during an input change but not simultaneously (one switches off **before** the other switches on), the OR briefly outputs `0` → **glitch / hazard** (an undesired short pulse).

| Path | Effect |
|---|---|
| AND2 turns on **before** AND1 turns off | OK (no glitch) |
| AND2 turns on **after** AND1 turns off | Glitch on output |

## Are glitches dangerous?

- For slow observers (LEDs, humans at 30 Hz), a 2 ps glitch is invisible.
- For digital systems sampling at high frequency, a glitch may be captured and produce a wrong state. We avoid them.

## Hazard mitigation via redundant terms

Add a redundant AND gate that holds the output at `1` during the switching transition. In K-Maps, the extra term covers the **interface between two adjacent sub-cubes**. Example:

```
F = a′c′ + cd       (optimised — but glitch-prone)
F = a′c′ + cd + a′d (with redundant term)
```

Caveats:

- Do **not** add redundant terms by default — only if asked (it conflicts with PPACT optimisation).
- This trick does not solve all glitch sources.
- Glitch avoidance is part of broader **Signal Integrity (SI)** issues (e.g., capacitive coupling between parallel wires on an IC; aggressor wire induces glitches in victim wire).

# 5. Introduction to sequential systems

## Limitation of combinatorial logic — the tank example

A digital system controls liquid in a tank with two sensors (S1 bottom, S2 top) and a fill valve `Vin` ( `Vout` is a random drain).

- **Tank full and being emptied** vs **tank empty and being filled**: same input combination (00), different desired output for `Vin`.
- A combinatorial circuit cannot distinguish — it would always produce the same output for the same input.

**We need memory** — to store in which **state** the system is.

## State sequence

Describe the system as a sequence of **states** `z = 1, 2, 3, 4` with the desired output controlling `Vin`:

- z = 1: tank full, inputs 00 → can be emptied → next state z = 2 if inputs 01.
- z = 2: emptying, Vin closed → next state z = 3 when inputs 00 (sensor1 reached, open valve).
- z = 3: filling, output O = 1 → next state z = 4 when inputs 01.
- z = 4: full, back to initial.

## Stable states, transitions

- A **stable state** persists as long as inputs do not change.
- A **transition** is triggered by an input change; takes the system from a present state to a future state.
- Once the transition completes, the future state becomes the present.

## Sequential systems — formal definition

In a sequential system the outputs and the next state both depend on inputs and current state:

```
O  = f₁(I, S⁻)
S⁺ = f₂(I, S⁻)
```

A sequential circuit is a combinatorial circuit with one or more **state-variable feedback loops**. State can be a vector (multiple Boolean values).

## Representing sequential systems

- **State graphs** — nice for small systems; states are circles, transitions are oriented arcs labelled with the input condition. Not preferred for synthesis (lacks systematic coverage).
- **State table / Huffman table / transition matrix** — preferred form. First column = present states; columns for all input combinations; last column for output(s).
- **Logic equations** — derived from the state table and used to build the circuit.

## State table conventions

- **Stable states** in **bold** in the cell where present = future for that input combination.
- **Transition states** are written in normal text (intermediate steps along a path between stable states).
- Non-existing transitions are marked with `–` (don't cares).

## State graph example

States S1–S4, two inputs a, b. Each arc is labelled with a Boolean condition (e.g., `a´b–` means a = 0 AND b = 1, c don't care).

## Reading a state table

Start in a stable state with given inputs. When inputs change, switch to the new column (same row): the cell value is the destination. If the destination is another stable state (bold), the system remains there. If a transition state, follow successively until reaching a stable state.

## Critical hypothesis

The number of transitions between two stable states is unbounded, but **inputs must NOT change during transitions** (transitions are not instantaneous). Otherwise, the system can end up in an unintended state.

---

# Part IV — Synthesis of sequential circuits (Th08)

## 1. Sequential systems — classes & representation

### Why sequential

Combinatorial circuits cannot compute outputs that depend on history. Sequential systems introduce **state** (a present that materialises the past), stored via feedback.

### Memory: present vs future

- The state is a feedback loop with a delay between the future signal $S^+$ and the present $S^-$.
- Time difference = wire/gate delay distinguishing $y$ (present) from $Y$ (future).

### Two classes of sequential systems

- **Moore machine** — output is a (combinatorial) function of state variables only.
- **Mealy machine** — output is a function of state variables AND inputs.

Choice has consequences for state optimisation and circuit complexity.

### Representations

- **State graphs** — visual, not used for synthesis.
- **State tables** — preferred for synthesis.
- **Logic equations** — for circuit construction.

## 2. State encoding

### Principle

State tables initially use arbitrary labels (decimal, letters); for digital implementation, each state must be encoded in **binary**.

- For $n$ states, use $\lceil \log_2 n \rceil$ state variables (1 bit each).
- Two binary state variables → exactly 4 states.

### Example

| State | Code |
|---|---|
| 1 | 00 |
| 2 | 01 |
| 3 | 11 |
| 4 | 10 |

The encoded state table maps each present-future pair to bit patterns. Each bit is a separate dedicated logic function (and a dedicated feedback loop).

### From encoded state table to K-Maps

For state vars `(y₂, y₁)` and inputs `(a, b)`, the encoded table directly becomes two K-Maps (one per state bit) plus K-Maps for outputs. Capital `Y` denotes the future-state input; lowercase `y` denotes the current-state output of the feedback memory.

## 3. Worked example — tank/reservoir Moore machine

### Procedure

1. Write the state table.
2. Draw the state graph.
3. Encode the states.
4. Derive K-Maps for state functions.
5. Derive output equations.
6. Enumerate all logic functions.
7. Draw the circuit diagram.

### State table for the tank

Initial state: tank full (inputs ab = 11). State 2 = emptying. State 3 = sensor1 reached, valve must open. State 4 = filling, between sensors. Don’t cares for ab = 10 (impossible because S2 above S1) and for instantaneous double-input changes.

| $S^+$ \ ab | 00 | 01 | 11 | 10 | Z |
|---|---|---|---|---|---|
| **1** | – | 2 | **1** | – | 0 |
| 2 | 3 | **2** | 1 | – | 0 |
| **3** | **3** | 4 | – | – | 1 |
| 4 | 3 | **4** | 1 | – | 1 |

### Encoding (1→00, 2→01, 3→11, 4→10) and K-Maps

- $Y_2 = a'b' + y_2a'$
- $Y_1 = a'b' + y_2'a'$
- $Z = y_2y_1' + y_2y_1 = y_2(y_1 + y_1') = y_2$ (state-decoder approach)

### Logic functions

```
Y₁ = f₁(a, b, y₁, y₂) = a′b′ + y₂′a′
Y₂ = f₂(a, b, y₁, y₂) = a′b′ + y₂a′
Z  = f₃(y₁, y₂) = y₂
```

The fact that some terms drop out does not change the Moore nature — the difference with Mealy lies in state-fusion possibilities.

### Final circuit

Two OR-NOR-AND combinations producing $Y_1$ and $Y_2$, each fed back as $y_1$, $y_2$. Output $Z$ is just a wire from $y_2$ (after optimisation).

## 4. Optimization of primitive state tables

### Synthesis steps recap

1. **Primitive state table (Huffman table)** — one stable state per line. Don't worry about redundancy; capture exact behaviour.
2. **State encoding** — choose unique binary codes per state.
3. **Logic equations** — derive optimised expressions from the encoded table.

## Why optimise

The complexity of the final circuit grows with the number of states `n` (more states → more bits → more functions and feedback loops). Reducing `n` reduces resources.

## State equivalence

Two stable states are **equivalent iff**:

1. They produce the same output (mandatory for Moore machines), AND
2. For all input combinations, all future states are the same or themselves equivalent.

Extends to ≥3 states: same outputs and all pairs equivalent two-by-two.

## Identical states & state fusion

- **Identical states** — same line entries, same outputs, same futures for the same input combinations.
- **Sate fusion** — stable states for **different** input combinations have the same future and same output.

## Fusing more than two states

To fuse states 1, 7, 8 (all with output 0, e.g.): need to verify that pairs (7, 8), (2, 4), … are all themselves equivalent. Use **state equivalence tables** to track conditions.

## Equivalence conditions table

Build a table with `n(n-1)/2` cells (each pair of states once). For each pair:

- **X** — not equivalent (e.g., different outputs in Moore machine).
- **OK** — directly equivalent.
- **Conditional** — equivalent iff some other pair is also equivalent.

After building, propagate constraints: if a referenced pair is X, the dependent pair becomes X too.

# 5. The D Flip-Flop — full synthesis example

## Specification

- Two inputs: `D` (data), `C` (control). One output `Q`.
- **Edge-triggered**: `Q` takes the value of `D` only on the **rising edge** of `C` (transition 0 → 1). Between rising edges, `D` variations are ignored.
- If `C` and `D` change simultaneously, `Q` keeps the **old** value of `D` (just before the rising edge).
- `D` must arrive **before** the rising edge of `C` (to avoid metastability).

Two variants:

- **Latches** — sensitive to the level of `C`.
- **Flip-flops** — sensitive to the **transition** (edge) of `C`.

## Primitive state table (10 states)

Built incrementally by analysing every input transition and creating a new state if needed (states 1–10). Output `Q` is `0` for stored 0; `1` for stored 1.

| $S^+$ \ CD | 00 | 01 | 11 | 10 | Q |
|---|---|---|---|---|---|
| **1** | **1** | 2 | 5 | 3 | 0 |
| **2** | 1 | **2** | 4 | 8 | 0 |
| **3** | 6 | 2 | 5 | **3** | 0 |
| **4** | 9 | 7 | **4** | 8 | 1 |
| **5** | 6 | 2 | **5** | 3 | 0 |
| **6** | **6** | 2 | 5 | 3 | 0 |
| **7** | 9 | **7** | 4 | 8 | 1 |
| **8** | 9 | 7 | 4 | **8** | 1 |
| **9** | **9** | 7 | 5 | 10 | 1 |
| **10** | 6 | 2 | 5 | **10** | 0 |

### State equivalence table — passes

After three passes (each refining conditional equivalences), find:

- Equivalent: 1 ≡ 6, 3 ≡ 10. Then deeper fusion possible: 1, 3, 5 → 1; 4, 7, 8 → 4; 2 → 2; 9 → 9.

### Final optimised state table (4 states)

| $S^+$ \ CD | 00 | 01 | 11 | 10 | Q |
|---|---|---|---|---|---|
| **1** | **1** | 2 | **1** | **1** | 0 |
| 2 | 1 | **2** | 4 | 4 | 0 |
| 4 | 9 | **4** | **4** | **4** | 1 |
| 9 | **9** | 4 | 1 | 1 | 1 |

Re-encoded with 1→00, 2→01, 3→11, 4→10:

K-Maps yield:

```
Y₂ = y₁C′ + y₂C
Y₁ = C′D + y₁C
Z (= Q) = y₂y₁ + y₂y₁′ = y₂
```

We moved from 10 → 4 states (4 → 2 state variables) — significant savings.

---

# Part V — Asynchronous sequential circuits (Th09)

## 1. Race conditions

### Problem definition

When inputs change, internal logic computes the next state. If two (or more) state variables must switch from `0` to `1` (or vice-versa) at the same time, **physical wire/gate delays** make one variable arrive before the other → the system may end up in an unintended (possibly stable) state.

- Hamming Distance between source $(y_2, y_1)$ and destination $(Y_2, Y_1)$ > 1 in the encoded state table → race condition.
- **Mandatory** to avoid race conditions for deterministic operation in asynchronous circuits.

### Race conditions — example

For a transition `00 → 11`, if $Y_1$ switches first, the system briefly sees `01`; if that is itself a stable state in the table, the system sticks there. The actual destination depends on which signal wins the race.

### Two solution classes

- **Asynchronous logic circuits** — make sure no transition has > 1 internal-variable change. Three methods (below).
- **Synchronous logic circuits** — use memories (Flip-Flops) to filter transient values; race conditions in the table become harmless. (Covered in Th10.)

### Race conditions in tables — visual

In the encoded state table, mark cells where present-to-future Hamming Distance > 1. Note: input changes can be instantaneous (they are random variables) and are not race conditions; only state-variable changes count.

## 2. Methods to solve race conditions

Apply in order; each later method introduces more constraints/cost.

### Method 1 — State encoding

- State encoding is free; perhaps a clever encoding is race-free.
- Use **state encoding graphs**: place each state at a vertex of an n-cube. Transitions are arcs; if any arc is a diagonal (HD > 1), there is a race.
- For 4 states on a 2-cube: edges = no race; diagonals = race. Try swapping codes (e.g., swap state IDs 2 and 3) to remove diagonals.
- Not always possible.

### Method 2 — Modifying transitions

Property: only source and destination matter, not the intermediate transitions.

- Use existing **don't cares** to route the transition through them, breaking it into single-bit changes.
- Or modify existing transitions: replace a direct `00 → 11` jump with `00 → 01 → 11`, borrowing an existing race-free intermediate.

### Method 3 — Adding an extra state variable

- If neither Method 1 nor 2 works, add a state variable $Y_3$.
- This doubles the K-Map size and introduces $2^n$ new transitions, all marked don't care.
- Use the new transitions (Method 2 style) to route races safely.
- **Caution**: solve all Method 2 races first; only the remaining ones use Method 3 to avoid over-constraining.

### Asynchronous synthesis recap

1. Establish the **primitive state table** from verbal specs.
2. State **reduction** (state optimisation).
3. State **encoding** (Method 1).
4. Solve remaining races (Method 2; Method 3 only if necessary).
5. Extract optimised state functions and synthesise outputs.

## 3. Example of asynchronous implementation

For the table:

| $S^+$ \ ab | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| **1** | **1** | 2 | 3 | 4 |
| **2** | 1 | **2** | 2 | 4 |
| **3** | 1 | 4 | **3** | 3 |
| **4** | 1 | 4 | 4 | **4** |

With default encoding 1→00, 2→01, 3→11, 4→10, the state encoding graph reveals races. Method 2 cannot fix the column ab = 11 race, so Method 3 ($Y_3$ added) is needed.

After adding $Y_3$, transitions are rerouted using `–` cells (e.g., `011 → 000` becomes `011 → 111 → 110 → 100 → 000`). However, blindly using new transitions over-constrains the K-Maps; preferably modify existing transitions where possible (e.g., the alternative encoding `1→00, 2→01, 4→11, 3→10` removes the need for an extra variable).

K-Maps for the extra-variable version yield:

```
Y₁ = y₂′a′b + y₁ab + y₂y₁b′ + y₃y₂
Y₂ = y₂b + y₂a + y₃a + y₁′ab′
Y₃ = y₂′y₁′ab + y₃y₁′
```

For the alternative encoding (preferred):

```
Y₂ = y₂a + y₂b + y₁′ab + y₁ab′
Y₁ = a′b + y₁b + y₂′ab′ + y₂y₁
```

# 4. Synthesis of Moore & Mealy outputs

## Moore output synthesis

Output is a sum of products (states) where output = 1: `Z = f(yᵢ) = y₂′y₁ + y₂y₁′`.

But this naive approach can produce **glitches** during transitions (e.g., 00 → 01 → 11 → 11 sequence yields 0 → 0 → 1 → 0; the unwanted `1` is a glitch).

### Single transition rule

To avoid output glitches, fix the output value during transitions so that **only one transition occurs from `0` to `1` or `1` to `0`**.

- If departure and destination outputs are the **same**: keep the output the same during the transition (no extra change).
- If departure and destination outputs **differ**: the transition can be left as **don't care** (one change anyway, occurring earlier or later).

### Multiple transitions

When Method 2 created multiple transition steps between two stable states, only **one** transition cell may be left as don't care; the rest must be fixed to enforce a single output change.

### Shared transitions

If a transition cell is shared by several stable-state paths (e.g., `2 → 3 → 3` and `1 → 2 → 3 → 3`), the output must be fixed by the most-constrained path; the other paths inherit.

### Worked Moore output synthesis example

| $S^+$ \ ab | 00 | 01 | 11 | 10 | Z |
|---|---|---|---|---|---|
| **1** | **1** | 2 | 2 | – | 0 |
| **2** | 1 | **2** | 3 | – | 1 |
| **3** | 3 | 2 | **3** | 4 | 0 |
| **4** | – | 4 | 3 | **4** | 1 |

After applying the single-transition rule, the K-Map yields:

```
Z = a´b + ab´    or    Z = a´b + y₂y₁´
```

### Mealy machine — definition revisited

In Mealy, the output depends on inputs too, so two stable states on the same row can have different outputs.

- This allows **fewer states after fusion** than Moore — potentially simpler implementations.
- But certain pairs cannot be fused: stable states in the **same column** with different outputs cannot be merged (the merged state would have ambiguous output for that input combination).

### Mealy state-table notation

Write outputs next to each state: `state/output`. One output column per input combination, instead of one global column.

| $S^+$ \ ab | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| **1** | **1/1** | 2 | 1/0 | |
| 2 | 2/0 | | | |

### Mealy output synthesis

Same rule: only one output transition between two stable states. With shared transitions, fix the most-constrained one first; subsequent ones may then be don't care.

## 5. Synthesis of Moore & Mealy machines — full example

### Spec

A primitive state table with 6 states and 2 inputs (a, b), as below:

| $S^+$ \ ab | 00 | 01 | 11 | 10 | Z |
|---|---|---|---|---|---|
| **1** | **1** | 2 | – | 3 | 0 |
| **2** | 1 | **2** | 4 | – | 1 |
| **3** | 1 | – | 4 | **3** | 0 |
| **4** | – | 5 | **4** | – | 1 |
| **5** | 6 | **5** | 4 | – | 0 |
| **6** | **6** | 5 | – | 3 | 1 |

### A. Moore optimisation

Equivalence-conditions table → only 1↔3 and 4↔6 fuse → final 4 states.

State encoding graph reveals one unavoidable race; fix with Method 2: `11 → 00` becomes `11 → 01 → 00`. Final state functions:

```
Y₂ = y₂a′ + ab
Y₁ = y₂′b + y₂b′ + ab
Z  = y₁
```

### B. Mealy optimisation

States 1↔6 and 2↔5 cannot fuse (stable states in same column with different outputs). Fusion graphs allow `1 ⊕ 2 ⊕ 3 → 1` and `4 ⊕ 5 ⊕ 6 → 2` → final 2 states. Encoded state functions:

```
Y₁ = y₁a′ + ab
Z  = ab + y₁′b + y₁a′b′
```

Mealy uses 2 circuits (instead of 3) but has a slightly more complex output. **Mealy is not systematically better.**

---

# Part VI — Synchronous logic circuits (Th10)

## 1. Race conditions & asynchronous circuits — recap and limits

### Recap

- Race conditions appear when more than one state variable must switch simultaneously (HD > 1).
- Asynchronous synthesis solves them with Method 1 → Method 2 → Method 3.
- Output synthesis uses the single-transition rule to avoid glitches.

### Asynchronous circuits — performance

- Each transition is a **system cycle**; stable-to-stable transitions can take several cycles.
- For an outside observer it is essential to wait the right number of cycles before reading outputs and never to change inputs during a transition.
- **Best possible performance**: each stable-state transition takes exactly the time needed.

### From asynchronous to synchronous

- Asynchronous time discretisation per transition becomes cumbersome at scale.
- Idea: **discretise time for the entire stable-state transition** — slow down all paths to match the slowest. Loss is acceptable if delay differences are small.

## 2. Synchronous logic circuits

### Controlled feedback delay → memory

- Insert a **memory element** in each state-feedback loop.
- All memories are updated by a common control signal — the **clock** `Clk`.
- The state update happens at regular (periodic) intervals.

### Synchronous operation

State change happens in two distinct steps:

1. **Compute the future state** from inputs and present state: $Y_i = f_i(I, y_i)$.
2. **Update memories**: $y_i \leftarrow Y_i$ on the active clock edge.

The Clk period must be long enough for all transients to settle at the memory inputs before the rising edge.

### Synchronisation behaviour

- Memories ignore (filter out) any transient values during the “next-state computation” interval.
- Memories act as a **synchronisation barrier** — fast paths wait for the slowest one.

### Performance

- The clock period is fixed by the **critical path** (slowest signal path).
- The system speed = how fast one stable state succeeds another, limited by this critical path.
- Sub-optimal vs asynchronous (which adapts), but acceptable when path delays are uniform.

### Memory elements = Flip-Flops (FFs)

Real circuits use FFs (SR, JK, D, T) as memory devices. The synchronous synthesis problem becomes: **synthesise the excitation functions** that drive each FF’s inputs to mimic the desired state-table behaviour.

- **SR** and **JK** need two excitation functions per state bit.
- **D** and **T** need only one.

## 3. Flip-Flops & sequential circuits synthesis

### Bistable circuit

Two cross-coupled inverters; output `Q` adopts whichever value establishes at power-up. Has two stable states ( `0` or `1` ) — hence **bistable** — but no inputs to change the value.

### Latches vs Flip-Flops

- **Latches** — sensitive to the **level** of the control input (output follows when control is active).
- **Flip-Flops** — sensitive to the **transition** (rising or falling edge) of the control input.
- For synchronous synthesis, we always use **edge-triggered Flip-Flops**, even if the clock is not explicitly drawn.

### State tables and graphs of the four FF types

| FF | $Q^+$ table |
|---|---|
| SR | 00→Q, 01→0, 11→ –, 10→1 |
| JK | 00→Q, 01→0, 11→Q′ (toggle), 10→1 |
| D | 0→0, 1→1 ($Q^+$ = D) |
| T | 0→Q (hold), 1→Q′ (toggle) |

State graphs of FF transitions show, for each `0 → 1` or `1 → 0` arc, the input combination that triggers it.

### SR latch with NAND gates

For the SR latch:

```
Q  = S + qR′ = (S′ · (q · R′))′
Q′ = R + q′S′ = (R′ · (q′ · S′))′
```

The `SR = 11` combination is **forbidden**: both `Q` and `Q′` go to `1`; the actual settled value depends on which gate switches first → unpredictable.

### Excitation tables

For synthesis, instead of encoded state tables, use **excitation tables** that describe how each excitation function must change to store the required value. Four operations exist:

| Op | Q | $Q^+$ | Meaning | Code |
|---|---|---|---|---|
| Maintain 0 | 0 | 0 | hold | $\mu_0$ |
| Enable | 0 | 1 | set | ε |
| Disable | 1 | 0 | reset | δ |
| Maintain 1 | 1 | 1 | hold | $\mu_1$ |

### Excitation tables per FF type

| Op | Q | $Q^+$ | S | R | J | K | D | T |
|---|---|---|---|---|---|---|---|---|
| $\mu_0$ | 0 | 0 | 0 | – | 0 | – | 0 | 0 |
| ε | 0 | 1 | 1 | 0 | 1 | – | 1 | 1 |
| δ | 1 | 0 | 0 | 1 | – | 1 | 0 | 1 |
| $\mu_1$ | 1 | 1 | – | 0 | – | 0 | 1 | 0 |

## 4. Synchronous logic circuits — synthesis algorithm

### Step-by-step

1. **Primitive state table** from verbal specs (one stable state per line; correctness essential).
2. **State table optimisation** (Moore: cannot fuse states with different outputs; Mealy: more fusion freedom).
3. **State encoding** — free choice; **don't fix race conditions** (the FFs filter them out).
4. **Excitation table** — derived from the encoded state table; independent of FF choice. Fill with $\mu_0$, $\mu_1$, δ, ε per state bit.
5. **Per-FF excitation functions** — depending on FF type (`SR, JK, D, T`), derive truth tables for each excitation input; minimise via K-Maps.
6. **Output synthesis** — revisit for synchronous specifics.

### Async vs sync synthesis — comparison

- Almost identical: synchronous skips the race-conditions step and adds excitation-function synthesis.
- The state table may have race conditions (harmless thanks to FFs).

### Output in synchronous Moore machine

- Output is a function of state variables (now coming from FFs) — clean signal.
- For multiple outputs, one may switch earlier than another → register each output in its own FF for fully synchronous, glitch-free output. Cost: **2 clock cycles latency** from input to output.

### Output in synchronous Mealy machine

- Output depends on inputs → glitches on inputs propagate to outputs through the combinatorial logic.
- Registering outputs again solves this (2 clock cycles latency, but `Z` valid for a full clock period).

### Reducing latency — next-state output

Synthesise the output as a function of the **future** state and inputs (instead of the present state). Best of both worlds: clean output and **1 clock cycle latency**. Caveat: critical path now spans both excitation and output logic.

### Registering inputs

Storing all inputs in FFs (powered by the same Clk) ensures fully synchronous operation. Inputs no longer "change during transitions" — they only change on clock edges.

### Pipelined / RTL

Multiple synchronous sub-circuits can be chained: outputs of one feed the inputs of the next. Since outputs are already registered, FFs at the next stage's input can be omitted. This is the **Register-Transfer Logic (RTL)** paradigm — used to design ICs with tens of millions of gates.

## 5. Example of synchronous sequential synthesis — rotation direction sensor

### Specification

A rotating disc divided into four black/white quadrants; two sensors detect colour ( `1` white, `0` black). From the sequence of colours, determine the rotation direction ( `Z = 1` clockwise, `Z = 0` counter-clockwise).

### Hypothesis

Sensor sampling is faster than disc rotation → instantaneous double-input changes (00↔11, 01↔10) **never happen**.

### Primitive state table (8 states)

Built incrementally:

| $S^+$ \ ab | 00 | 01 | 11 | 10 | Z |
|---|---|---|---|---|---|
| **1** | **1** | 2 | – | 3 | 1 |
| **2** | 1 | **2** | 4 | – | 0 |
| **3** | 5 | – | 6 | **3** | 1 |
| **4** | – | 7 | **4** | 8 | 0 |
| **5** | **5** | 2 | – | 3 | 0 |
| **6** | – | 7 | **6** | 8 | 1 |
| **7** | 1 | **7** | 4 | – | 1 |
| **8** | 5 | – | 6 | **8** | 0 |

Two halves: clockwise sequence `1, 3, 6, 7`; counter-clockwise sequence `2, 4, 8, 5`.

## A. Moore optimisation

Equivalence table after several passes shows **no fusion possible** → too complex; abandon Moore.

## B. Mealy optimisation

Pairs that cannot fuse are those with different outputs in the same column. Fusion candidates: `1 ⊕ 2 → 1`, `3 ⊕ 5 → 2`, `4 ⊕ 7 → 3`, `6 ⊕ 8 → 4`. Result: 4 final states.

### Optimised Mealy table

| $S^+$ \ ab | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| **1** | **1/1** | **1/0** | 3 | 2 |
| **2** | **2/0** | 1 | 4 | **2/1** |
| **3** | 1 | **3/1** | **3/0** | 4 |
| **4** | 2 | 3 | **4/1** | **4/0** |

### State encoding (1→00, 2→01, 3→11, 4→10)

K-Maps yield:

```
Y₂ = y₂b + y₂a + ab
Y₁ = ab′ + y₁a + y₁b′
Z  = y₂y₁′a′ + y₂y₁b + y₂′y₁a + y₂′y₁′b′
```

## C. Synchronous JK implementation

State encoding (default), preserve race conditions:

| $Y_2Y_1$/Z \ ab | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 00/1 | 00/0 | 11 | 01 |
| 01 | 01/0 | 00/0 | 10 | 01/1 |
| 11 | 00 | 11/1 | 11/0 | 10 |
| 10 | 01 | 11 | 10/1 | 10/0 |

### Excitation table (per memory)

For each cell, look at `(y → Y)` to label with `μ₀, μ₁, δ, ε`. Then split per FF (M2 for $y_2$, M1 for $y_1$).

### Per-FF JK truth tables (using JK excitation values from the table)

After K-Map minimisation:

```
J₂ = ab            K₂ = a′b′
J₁ = y₂a′ + y₂′a   K₁ = y₂′b + y₂b′
```

### Output

`Z = y₂′y₁′b′ + y₂′y₁a + y₂y₁a′ + y₂y₁′b`

### Complete schematic

Two JK Flip-Flops (M1 storing $y_1$, M2 storing $y_2$), driven by the four excitation functions above, with a common Clk and the Z output combinatorial logic on top of `(y₂, y₁, a, b)`.

## D. Synchronous D-FF implementation

Using the D excitation values ( `μ₀ → 0, ε → 1, δ → 0, μ₁ → 1` ):

After K-Map minimisation:

```
D₂ = ab + y₂b + y₂a´
D₁ = y₂´y₁´a + y₂´y₁b´ + y₂y₁b + y₂y₁´a´
Z  = y₂´y₁´b´ + y₂´y₁a + y₂y₁a´ + y₂y₁´b
```

## Side-by-side comparison

| Implementation | State equations | Output |
|---|---|---|
| **A — Moore** | abandoned (too complex) | — |
| **B — Mealy asynchronous** | $Y_2 = y_2b + y_2a + ab$ ; $Y_1 = ab' + y_1a + y_1b'$ | $Z = y_2y_1'a' + y_2y_1b + y_2'y_1a + y_2'y_1'b'$ |
| **C — Mealy synchronous (JK)** | $J_1 = y_2a' + y_2'a$ ; $K_1 = y_2'b + y_2b'$ ; $J_2 = ab$ ; $K_2 = a'b'$ | $Z = y_2'y_1'b' + y_2'y_1a + y_2y_1a' + y_2y_1'b$ |
| **D — Mealy synchronous (D)** | $D_2 = ab + y_2b + y_2a'$ ; $D_1 = y_2'y_1'a + y_2'y_1b' + y_2y_1b + y_2y_1'a'$ | $Z = y_2'y_1'b' + y_2'y_1a + y_2y_1a' + y_2y_1'b$ |

## Exam note

Because of strong dependencies between synthesis steps, sequential synthesis is split into independent exercises:

1. Primitive state table construction.
2. State table optimisation for Moore and Mealy.
3. Excitation function and output synthesis from the optimised state table.

When implementing **synchronous** sequential circuits, do **not** solve race conditions — leave them in the encoded state table.

---

# Cross-cutting summary — the unified synthesis pipeline

## End-to-end flow (combinatorial → asynchronous → synchronous)

1. **Number representation** (Th05) — pick a base; for negative numbers, use **C2** in any modern system.
2. **Boolean algebra** (Th05) — manipulate logic equations using axioms and theorems (commutativity, associativity, distributivity, De Morgan, absorption, consensus, idempotency, involution, duality).
3. **Logic-function representation** (Th06) — TT, algebraic expressions (CDNF/SoM, CCNF/PoM), schematics. Build canonical forms; mention NAND/NOR are singleton FCSs.
4. **Optimisation** (Th06) — K-Maps for ≤ 5 vars (PIs, EPIs, “smallest number of biggest sub-cubes”, don’t cares for inputs and outputs).
5. **Combinatorial synthesis** (Th07) — verbal spec → TT → optimised expressions → circuit; multiple outputs via concurrent combinatorial circuits; manage hazards with redundant terms (only when asked).
6. **Sequential introduction** (Th07) — when a combinatorial machine cannot distinguish situations (e.g., tank example), introduce **state**, feedback, present ( `y` ) vs future ( `Y` ).
7. **Sequential synthesis** (Th08) — primitive (Huffman) state table → state-equivalence and fusion (more freedom for Mealy than Moore) → state encoding → K-Maps for next-state functions → outputs (state decoder).
8. **D Flip-Flop** (Th08) — full synthesis derived from a sequential spec; demonstrates that bistables themselves are designed using the same tools.
9. **Asynchronous logic** (Th09) — race-free state encoding (Method 1), don’t-care or transition-modification routes (Method 2), extra state variable (Method 3, last resort). Output synthesis uses the **single-transition rule** to avoid glitches.
10. **Synchronous logic** (Th10) — insert FFs (SR, JK, D, T) in the state feedback; common Clk filters races; synthesis becomes finding **excitation functions** per FF; outputs may be registered (clean, +1 cycle latency) or computed from future state (clean, no extra latency, longer critical path).

## Moore vs Mealy at a glance

| Feature | Moore | Mealy |
|---|---|---|
| Output formula | `Z = f(yᵢ)` | `Z = f(yᵢ, I)` |
| State fusion | only same-output states | also different-output states (subject to no-conflict-in-column rule) |
| Typical state count | larger | smaller |
| Output complexity | simpler | possibly larger |
| Glitches on input | shielded | propagate to output (use registered outputs) |

## Asynchronous vs synchronous at a glance

| Feature | Asynchronous | Synchronous |
|---|---|---|
| Race conditions | must be removed (Methods 1–3) | tolerated (FFs filter) |
| Time per transition | varies per state pair | fixed (one Clk period, ≥ critical path) |
| Memory elements | none beyond wires/ feedback | FFs (SR, JK, D, T) |
| Output glitches | combat with single-transition rule | combat by registering outputs or using next-state output logic |
| Performance | best-case (per-transition) | worst-case (critical path) but uniform |
| Design simplicity | harder (many corner cases) | easier and scalable (basis of RTL) |

## Useful identities and rules to remember

- **De Morgan**: `(x + y)′ = x′y′`, `(xy)′ = x′ + y′`.
- **Consensus**: `xy + x′z + yz = xy + x′z`.
- **Term expansion** to recover minterms: `xy′ = xy′(z + z′)`.
- **C2 negation**: invert all bits, add 1; subtraction = addition.
- **Hamming distance** between adjacent K-Map cells = 1 (Gray-code ordering of rows/cols).
- **Largest sub-cube** has size `2^k` and removes `k` variables.
- **Excitation operations**: $\mu_0$ (hold 0), ε (set), δ (reset), $\mu_1$ (hold 1).
- **Don't cares** can be `0` or `1` whichever helps; do not blindly set to `1`.
- **In sync synthesis, leave race conditions in the table.**
- **In async synthesis, fix the output during transitions to a single value change.**
- **Single FCS gates**: NAND or NOR alone suffice to build any logic function.